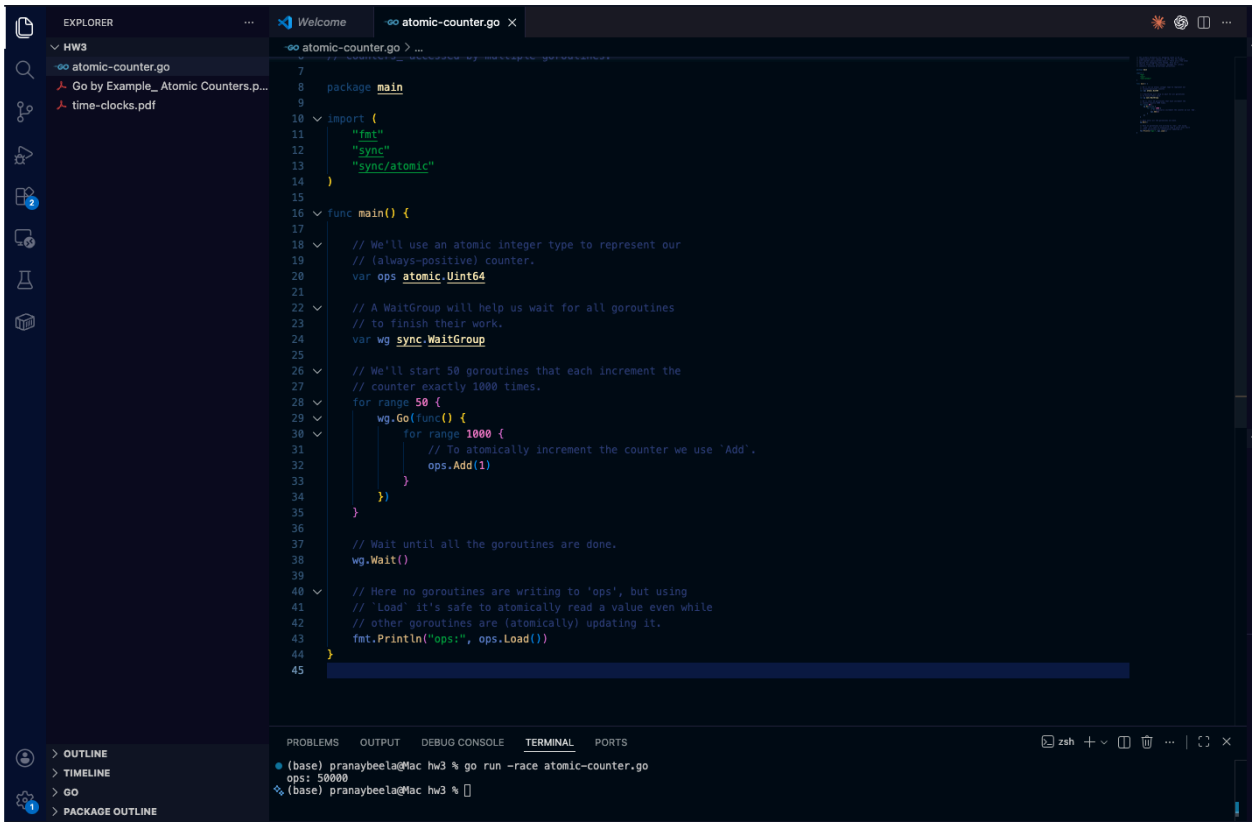
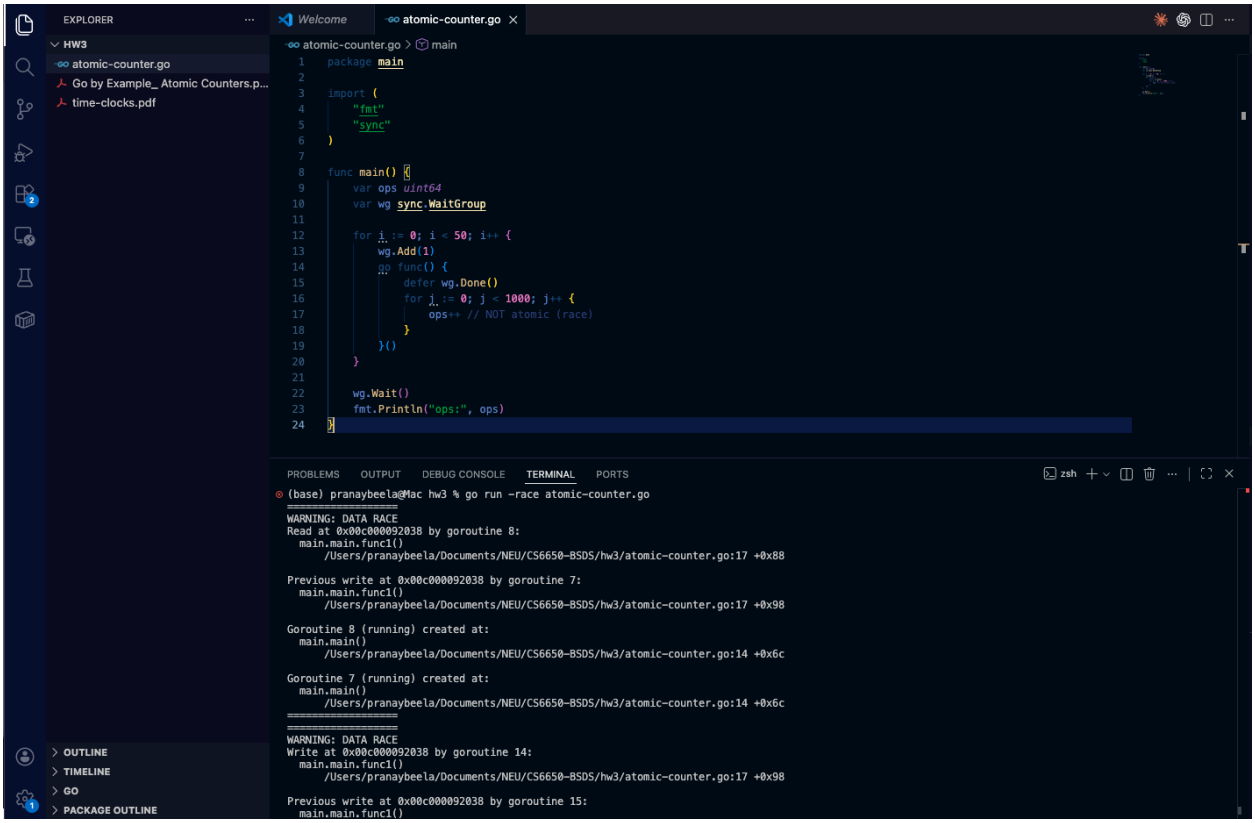
Atomicity

With atomic (ops.Add(1)): we will always see the result to be 50000 because each increment is thread-safe/atomic.

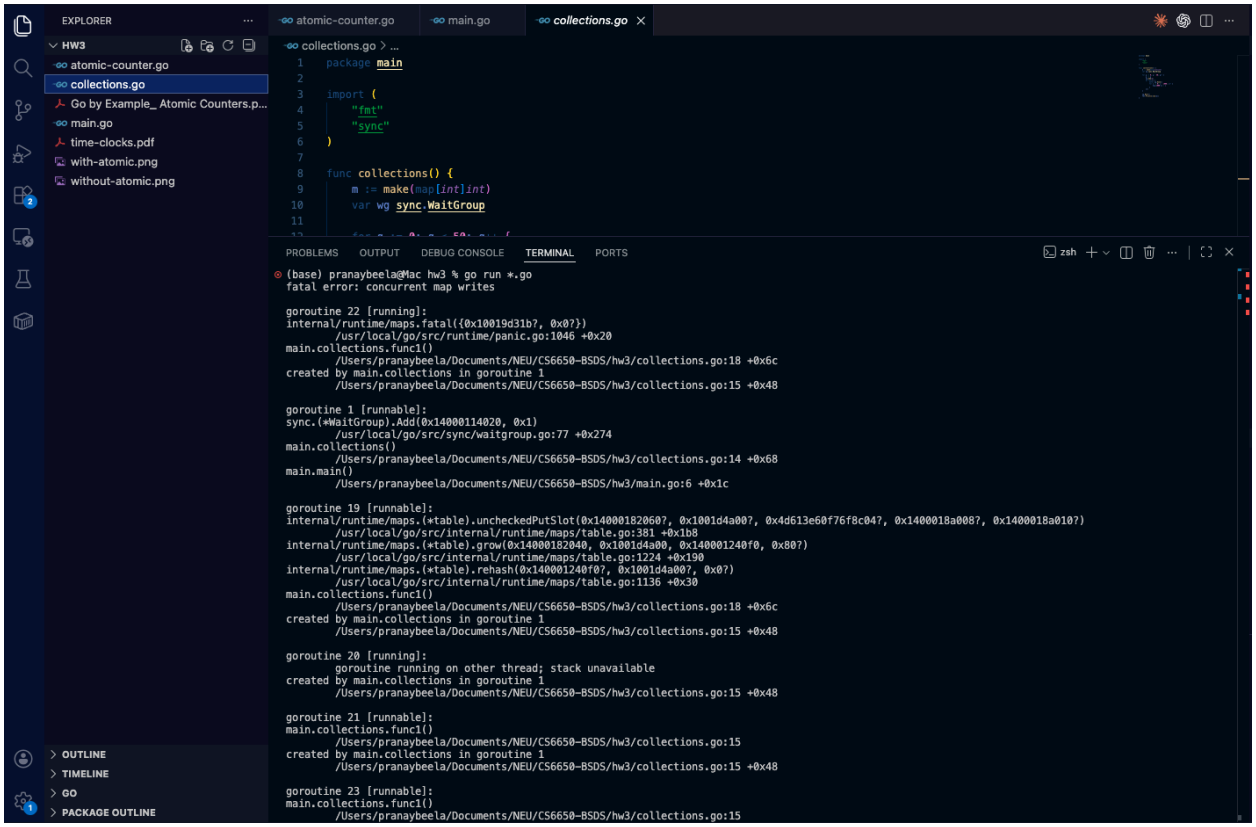


Without atomic (ops++): we will often see a different number (usually < 50000) and -race prints DATA RACE because multiple goroutines are reading/writing ops at the same time which implies that there would be updates lost.

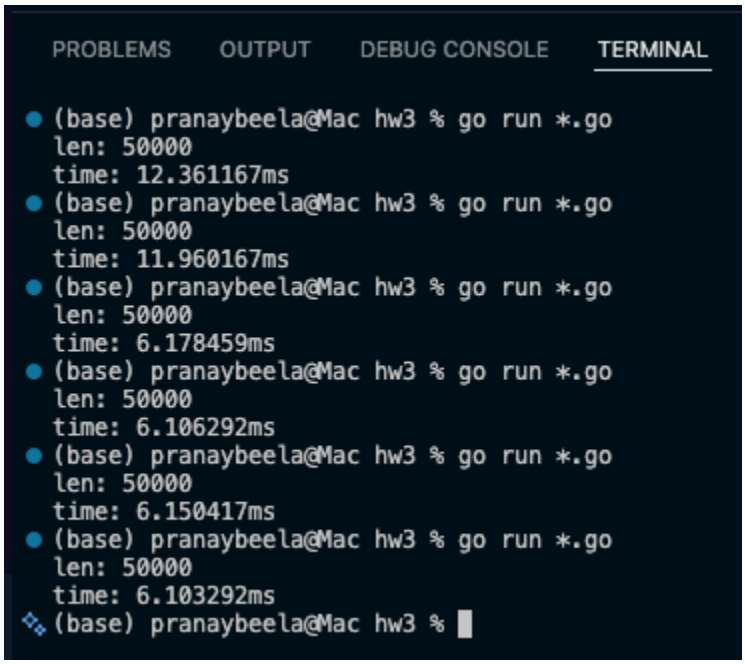


Collections

Without Mutex: Concurrent map writes means multiple goroutines are writing to the same Go map at the same time. Plain map is not thread-safe, so Go crashes to prevent corruption even if the keys are different.



With Mutex: A plain Go map is not safe for concurrent access, so we must synchronize, we are doing that using ***sync.Mutex*** here. A mutex prevents concurrent map access, so results are correct, here, len=50000. The trade-off is it makes updates happen one at a time, reducing parallelism and slowing things down.

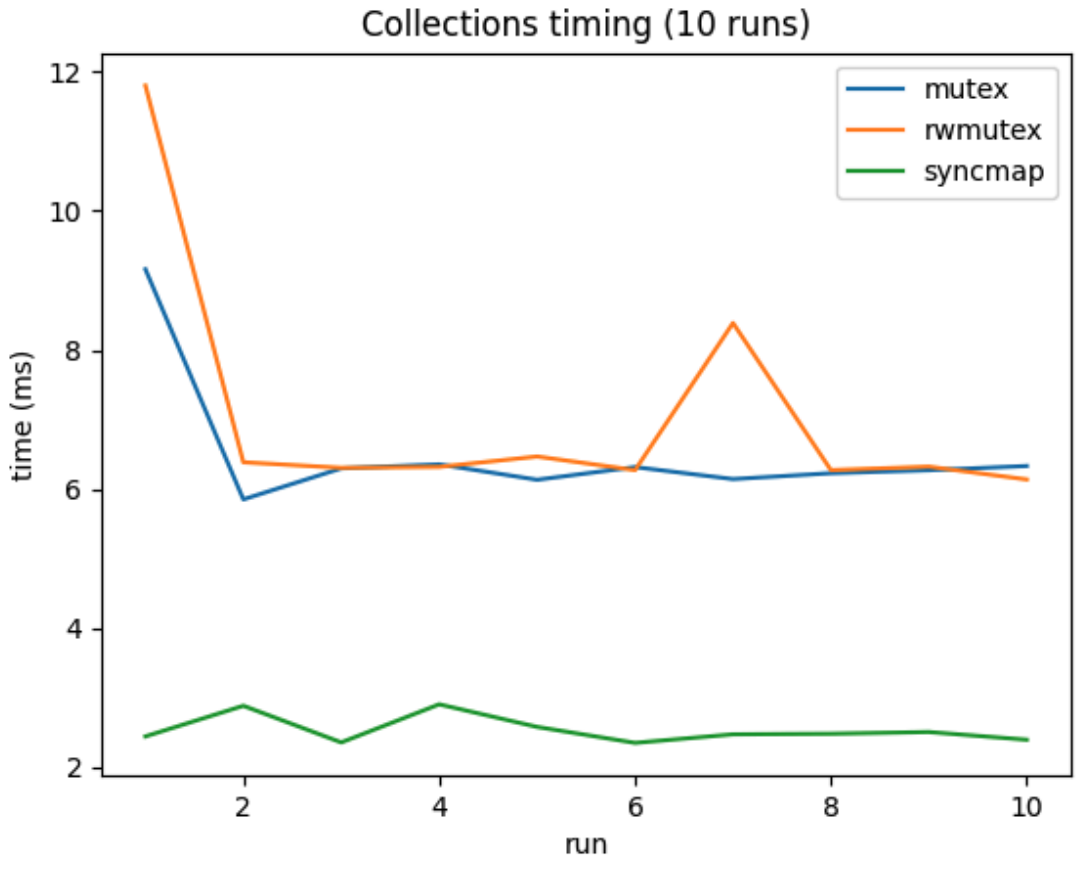


With RWMutex:

With Sync.Map:

```
(base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 4.035708ms
(base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 3.331625ms
(base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 3.573958ms
(base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 4.040709ms
(base) pranaybeela@Mac hw3 %
```

Comparing the run times:



Why these results?

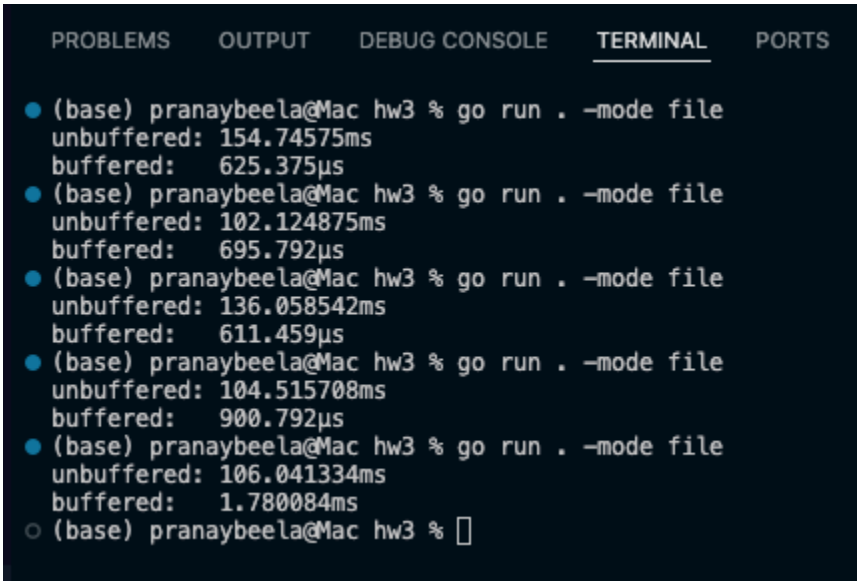
Every ***m[k]=v*** mutates a shared state, so ***Mutex*** and ***RWMutex*** force one writer at a time, creating contention and similar runtimes. ***sync.Map*** is designed for concurrent access and reduces lock contention internally, so it's often faster here.

If reads dominate: ***RWMutex*** can win vs ***Mutex*** because many goroutines can hold ***RLock*** at once (parallel reads). ***Mutex*** still allows only one reader or writer at a time, so it bottlenecks.

Tradeoffs:

1. **Mutex:** Simplest, type-safe ***map[K]V***, good general default; can bottleneck under high concurrency.
2. **RWMutex:** Best when there are a lot of reads and few writes. A write still blocks all readers and writers, and ***RWMutex*** adds overhead can be slower if not read-heavy.
3. **sync.Map:** Easiest safe concurrent map without manual locks. Often used strong for high concurrency and read-heavy or stable keys,but is less type-safe because it stores values as any, it uses a different API like Store, Load, and Range, and it is not guaranteed to be the fastest choice for every workload, especially when writes are very frequent.

File Access:



Why: unbuffered does a write call per line -tons of expensive OS work. Buffered batches many lines in memory and writes in bigger chunks - far fewer OS calls.

Tradeoff: buffered needs Flush()/Close() to guarantee data is written; it uses extra memory and can lose the last buffered chunk if the program crashes before flushing, but it's much faster

Context Switching:

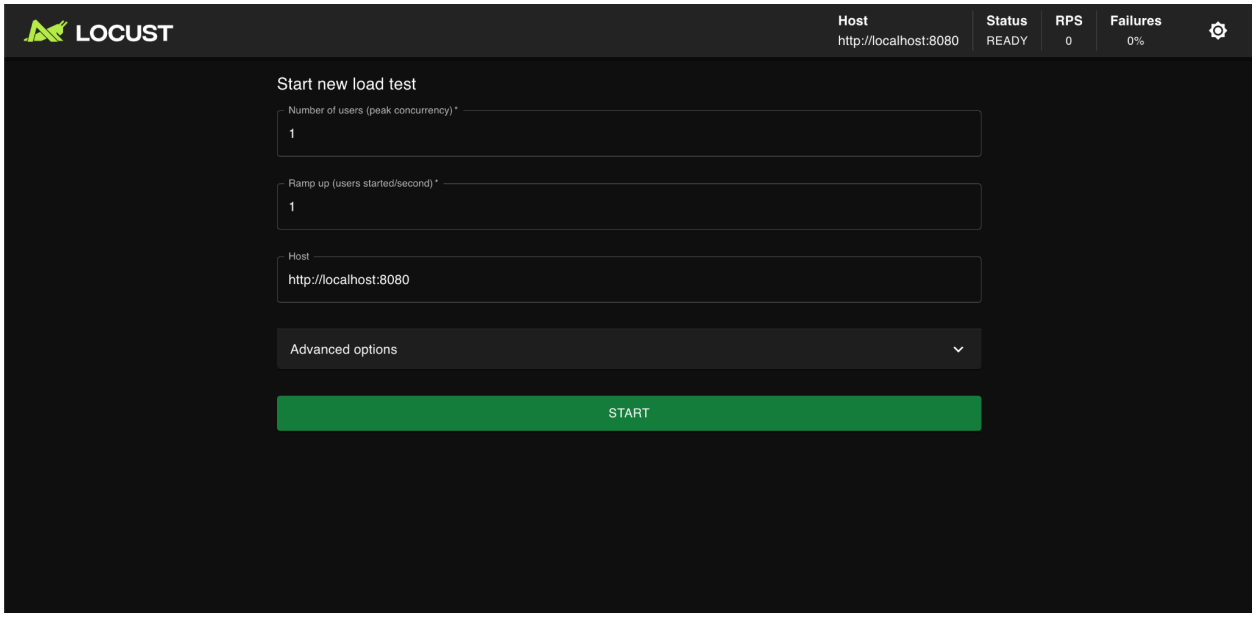
- GOMAXPROCS=1 avg handoff is ~76–94 ns (total ~153–222 ms)
- GOMAXPROCS=all avg handoff is ~110–111 ns (total ~221–223 ms)

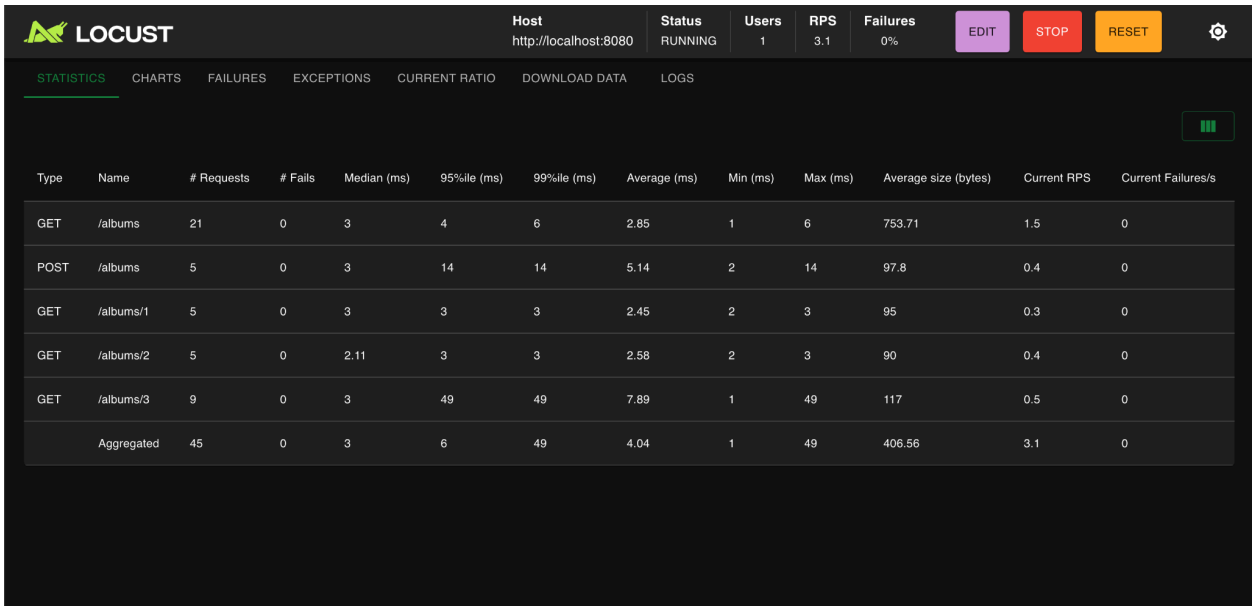
Which is faster? GOMAXPROCS=1 is faster here.

Why? With 1 OS thread, both goroutines stay on the same thread, so the scheduler and cache behavior are simpler. With multiple threads, the goroutines can bounce across threads/cores, adding extra scheduling + cache/coherency overhead for the channel handoff.

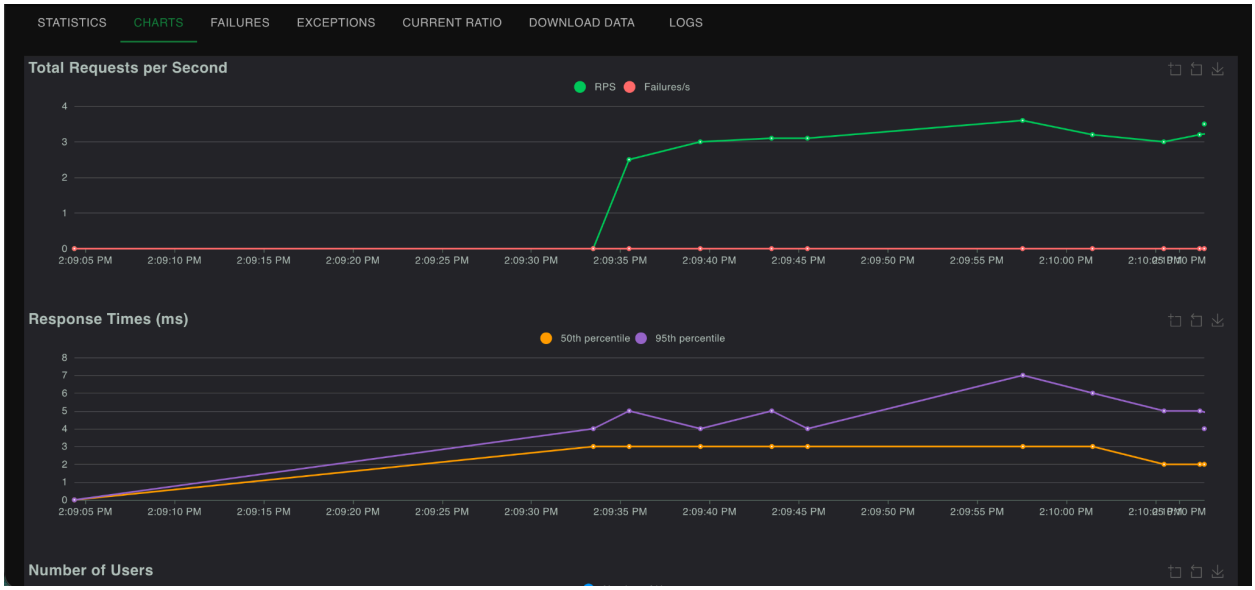
Relation to processes/containers/VMs: goroutine context switches are the cheapest; switching between OS threads/processes is heavier, and VMs are heavier still because there's more state and isolation involved.

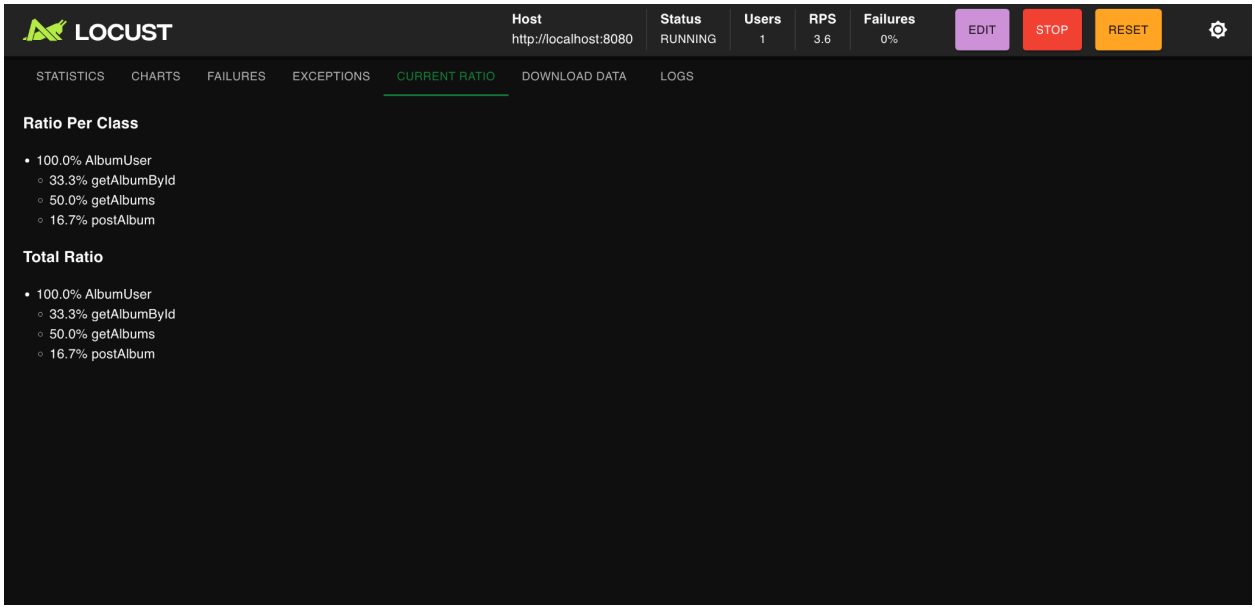
Locust:

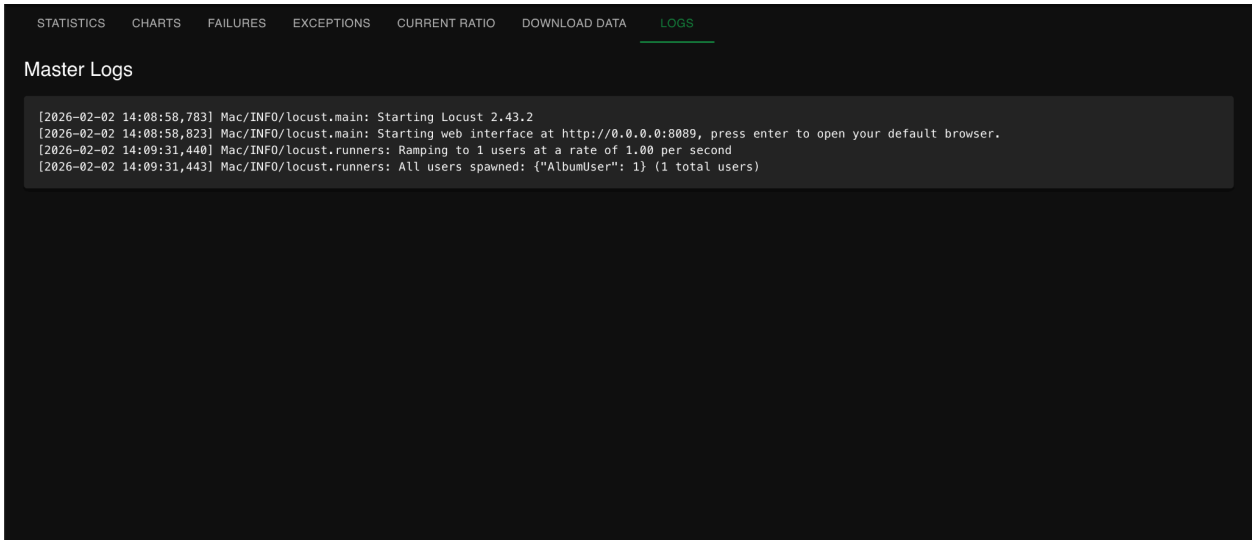




Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	21	0	3	4	6	2.85	1	6	753.71	1.5	0
POST	/albums	5	0	3	14	14	5.14	2	14	97.8	0.4	0
GET	/albums/1	5	0	3	3	3	2.45	2	3	95	0.3	0
GET	/albums/2	5	0	2.11	3	3	2.58	2	3	90	0.4	0
GET	/albums/3	9	0	3	49	49	7.89	1	49	117	0.5	0
	Aggregated	45	0	3	6	49	4.04	1	49	406.56	3.1	0

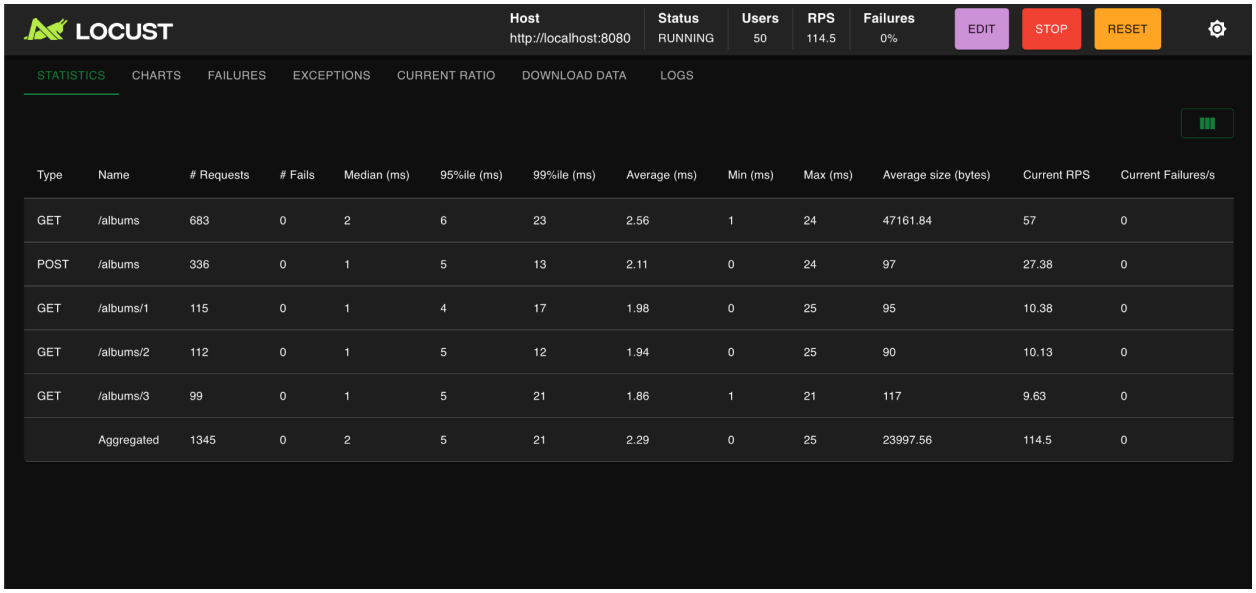

STATISTICS
CHARTS
FAILURES
EXCEPTIONS
CURRENT RATIO
DOWNLOAD DATA
LOGS
Total Requests per Second
RPS
Failures/s
4
3
2
1
0
2:09:05 PM
2:09:10 PM
2:09:15 PM
2:09:20 PM
2:09:25 PM
2:09:30 PM
2:09:35 PM
2:09:40 PM
2:09:45 PM
2:09:50 PM
2:09:55 PM
2:10:00 PM
Response Times (ms)
50th percentile
95th percentile
8
7
6
5
4
3
2
1
0
2:09:05 PM
2:09:10 PM
2:09:15 PM
2:09:20 PM
2:09:25 PM
2:09:30 PM
2:09:35 PM
2:09:40 PM
2:09:45 PM
2:09:50 PM
2:09:55 PM
2:10:00 PM
Number of Users


LOCUST
Host
http://localhost:8080
Status
RUNNING
Users
1
RPS
3.6
Failures
0%
EDIT
STOP
RESET
STATISTICS
CHARTS
FAILURES
EXCEPTIONS
CURRENT RATIO
DOWNLOAD DATA
LOGS
Ratio Per Class
100.0% AlbumUser
33.3% getAlbumById
50.0% getAlbums
16.7% postAlbum
Total Ratio
100.0% AlbumUser
33.3% getAlbumById
50.0% getAlbums
16.7% postAlbum

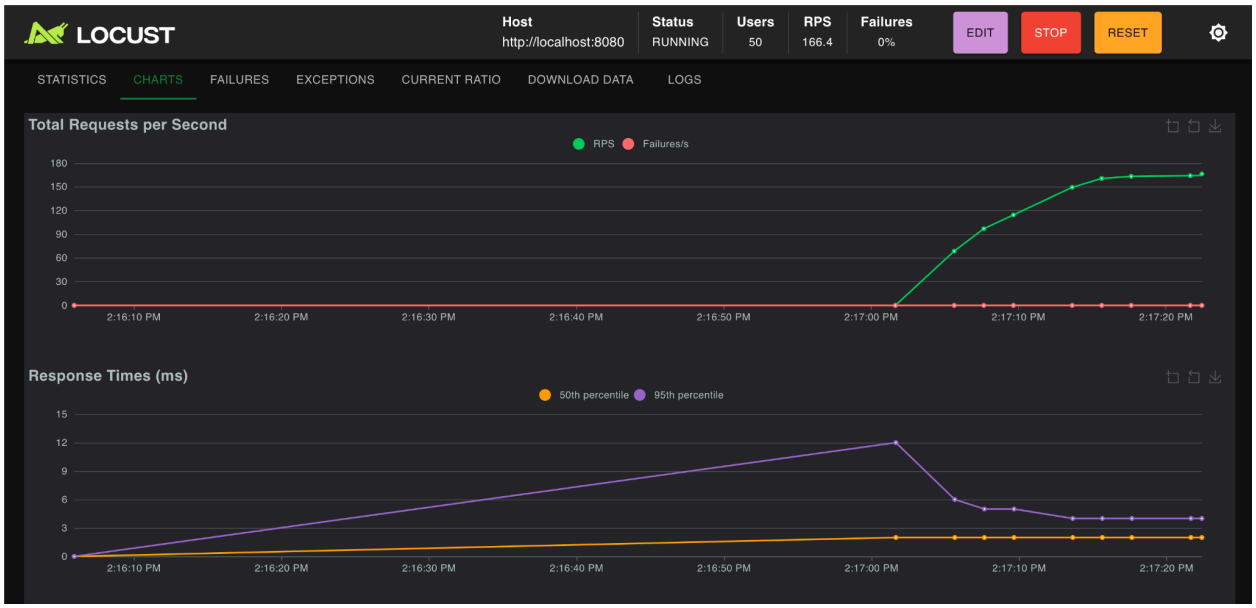


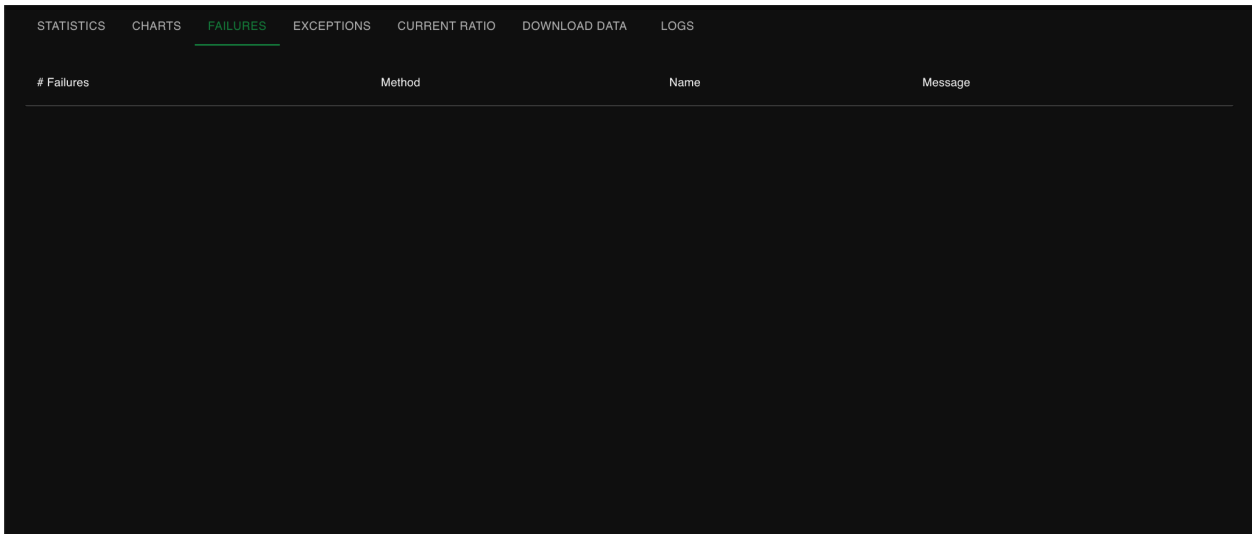
- What's going on is simply that different endpoints do different amounts of work, so their latency/RPS differ. GET /albums and GET /albums/:id are mostly read + JSON serialization, so they're usually faster and can hit higher RPS.
- **POST /albums does extra work:** parse JSON (BindJSON), validate/allocate, append to the shared slice, and return a response. That overhead makes it slower and lowers RPS.
- **Why the numbers vary even within GETs:**
    - **Payload size differs:** /albums returns the whole list, /albums/:id returns one item, different JSON size and serialization time.
    - Tail latency spikes happen from Go runtime/OS scheduling, GC pauses, and background CPU noise, so p95/p99/max can jump even with 1 user.
    - Your task weights control how often each endpoint is hit, so "Aggregated" stats reflect the mix, not a single route.

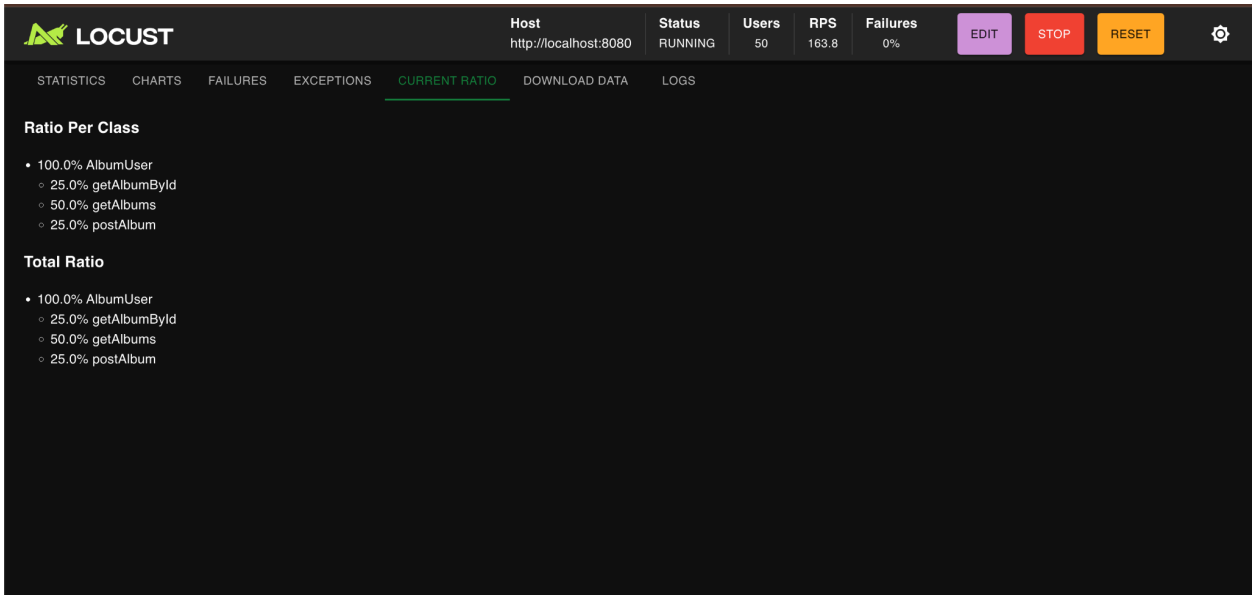
Local Test:



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	683	0	2	6	23	2.56	1	24	47161.84	57	0
POST	/albums	336	0	1	5	13	2.11	0	24	97	27.38	0
GET	/albums/1	115	0	1	4	17	1.98	0	25	95	10.38	0
GET	/albums/2	112	0	1	5	12	1.94	0	25	90	10.13	0
GET	/albums/3	99	0	1	5	21	1.86	1	21	117	9.63	0
	Aggregated	1345	0	2	5	21	2.29	0	25	23997.56	114.5	0







Setup: 1 worker, 50 users, ramp-up 10 users/sec. Task mix is ~3:1 GET:POST (mostly reads).
Results: Total throughput is about 115–165 RPS with 0% failures. Latencies stay low: median around 1–2 ms, p95 around ~5 ms, p99 around ~13–23 ms (some tail spikes).
Why these numbers: GETs are mostly reading + JSON serialization, so they stay fast. POSTs require JSON parsing and updating shared state, so they contribute more overhead and can increase tail latency.

Amdahl's Law:

- GET vs POST throughput differences make sense because GET mostly reads + serializes JSON, while POST also parses JSON and updates shared state.
- Adding more workers/users will increase throughput, but it won't scale linearly because some parts stay serial or contended (server CPU, shared data updates, GC/runtime overhead, network/HTTP overhead).
- If multiple requests read/write the same shared structure (like a map/slice), contention/races/locking can limit scaling and increase tail latency.

Context Switching:

- FastHttpUser reduces client-side overhead compared to HttpUser, so Locust can generate higher RPS with lower latency.
- After switching, the bottleneck usually moves to the server (CPU, JSON serialization, shared-state contention), so improvements plateau once the server is saturated.